

SIMATS ENGINEERING

ASSESSMENT TOOL 2

**COURSE CODE: CSA14
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:100

Annexure A

CASE STUDY

Code of Conduct:

I P.Karpaga shree(192424042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Karpaga shree

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

CASE STUDY

Set 3

1. A software company is developing a mini programming language for scientific calculations, where arithmetic expressions must be translated into syntax trees before code generation. During compilation, the compiler team notices that inherited and synthesized attributes are not being evaluated correctly for nested expressions. Initially, they use S-attributed definitions for bottom-up evaluation but later require L-attributed definitions to support function parameters and variable scopes. The compiler must also perform semantic checks to ensure that integer and floating-point operations are compatible. To improve reliability, the team plans to integrate automatic type conversion wherever necessary.

 - a) Explain how syntax-directed definitions help in syntax tree construction.
 - b) Compare S-attributed and L-attributed definitions in this scenario.

2. A student designing a compiler for an educational programming language wants the parser to generate intermediate code directly during parsing. The compiler initially uses top-down translation schemes for simple assignment statements, but difficulties arise when handling nested loops and expressions with multiple operators. To simplify semantic analysis, syntax trees are constructed and evaluated using bottom-up methods. During testing, several type mismatch errors occur because variables of different types are used together in expressions. The student decides to implement a simple type checker with automatic type conversions.

 - a) Why are syntax trees important in translation?
 - b) How does a type checker improve compiler reliability?

3. A robotics software company is building a domain-specific language where commands are translated into machine instructions using syntax-directed translation. The compiler team uses bottom-up evaluation of semantic rules to generate syntax trees and intermediate representations efficiently. However, problems arise when expressions contain incompatible data types such as integers, booleans, and floating-point values. To address this issue, the team develops a type system capable of checking type equivalence and applying suitable type conversions automatically. They also compare top-down and bottom-up translation techniques to improve overall compiler performance.

 - a) Explain the role of type systems in compiler design.
 - b) How does bottom-up evaluation assist syntax-directed translation?

4. In a compiler laboratory project, a team is implementing semantic analysis for a calculator-like programming language. The parser successfully recognizes expressions, but the compiler fails to generate correct syntax trees for complex statements involving nested arithmetic operations. To solve this issue, the students implement syntax-directed definitions using synthesized and inherited attributes. During execution, the compiler frequently reports

errors because variables with incompatible type expressions are compared and assigned incorrectly. The team then introduces type equivalence checking and implicit type conversion mechanisms to improve semantic correctness.

- a)** What is the significance of syntax-directed definitions in semantic analysis?
 - b)** How does type equivalence help prevent semantic errors?
-

- 5.** A startup developing a scripting language for data analysis needs a compiler that can translate high-level statements into optimized intermediate code. The compiler initially uses top-down translation methods for expression evaluation but struggles with handling complex attribute dependencies in nested statements. To improve efficiency, the development team adopts bottom-up evaluation techniques along with syntax tree construction. During semantic analysis, the compiler identifies several type inconsistencies between arrays, integers, and floating-point variables. As a solution, the team designs a simple type checker capable of verifying type expressions and performing safe type conversions.

- a)** Compare top-down and bottom-up translation methods.
- b)** Why are type checking and type conversion essential in modern compilers?

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CASE STUDY

SET 3: ANSWERS

Question 1

a) How Syntax-Directed Definitions help in syntax tree construction

A Syntax-Directed Definition (SDD) attaches a set of attributes and semantic rules to every production of the grammar. Each grammar symbol carries attributes — synthesized attributes, which are computed from the attribute values of its children, and inherited attributes, which are computed from the attribute values of its parent and siblings. For the scientific-calculation language in this case, an SDD can be written for the expression grammar so that every reduction also builds a node of the syntax tree, for example $E \rightarrow E_1 + E_2$ { $E.\text{node} = \text{mknode}('+', E_1.\text{node}, E_2.\text{node})$ }. Here the synthesized attribute node of E is built directly from the nodes already computed for E1 and E2.

Because a syntax tree keeps only the operators and operands that matter for translation and discards the extra grammar structure that a parse tree retains, using an SDD in this way lets the compiler build a compact tree instead of a bulky parse tree, directly reducing memory use, which matches the company's concern with translating expressions efficiently into intermediate code.

For nested expressions specifically, the SDD's underlying attribute-dependency graph fixes the order in which rules must be evaluated: the attributes of the inner sub-expressions are always evaluated before the attributes of the enclosing expression, so a node for an inner operation is always available before it is linked into its parent node. This is precisely what was failing in the company's compiler — once the dependency graph is respected (first with S-attributed rules, and later extended with inherited attributes), nested arithmetic expressions are translated into correctly structured syntax trees, and the annotated tree that results carries all the information the later code-generation phase needs.

b) Comparing S-attributed and L-attributed definitions in this scenario

S-attributed definitions use only synthesized attributes, so every attribute value is computed strictly from the values of the child nodes, and evaluation proceeds bottom-up in the same order a bottom-up (LR) parser reduces the input. This is exactly what the compiler team used first: for straightforward nested arithmetic expressions, the value/type/node of each sub-expression could be synthesized directly from its operands as each production was reduced, which is simple to implement and integrates naturally with a bottom-up parser.

The limitation the team encountered is that S-attributed definitions cannot pass information downward — they cannot tell a sub-tree what its expected type is, or which scope/parameter list it belongs to. This is why L-attributed definitions were required once function parameters and variable scopes had to be supported: an L-attributed definition allows both synthesized and inherited attributes, but restricts inherited attributes so that a symbol on the right of a production can depend only on the inherited attributes of the parent and on the attributes of symbols to its left in the same production, never on symbols to its right. This restriction still allows a single left-to-right, depth-first evaluation pass, so scope information and parameter types can be pushed down into function calls correctly while the tree is being built.

Basis	S-attributed Definitions	L-attributed Definitions
Attribute types used	Only synthesized attributes	Synthesized and inherited attributes (restricted)
Evaluation order	Bottom-up, in postorder, as each production is reduced	Left-to-right, depth-first (top-down or during a bottom-up parse with a stack of inherited values)
Parser suitability	Naturally fits LR / bottom-up parsers	Fits LL / top-down parsers, and can be adapted to LR parsing
Information flow	Values flow only upward from children to parent	Values can flow downward from parent/left-siblings as well as upward
Use in this scenario	Used first to build the syntax tree for simple nested arithmetic expressions	Adopted later to pass parameter types and scope/context information into function calls
Limitation	Cannot pass contextual information such as scope or parameter type down the tree	Evaluation is slightly more complex to implement than pure S-attributed schemes

In short, S-attributed definitions were sufficient for the simple, self-contained arithmetic expressions the language began with, while L-attributed definitions became necessary once contextual information such as parameter types and scope had to flow into nested sub-expressions — and this same inherited-attribute mechanism is what allows the type of an expression to be checked and, where necessary, automatically converted between integer and floating-point operands, which was the reliability goal the team wanted to achieve.

Question 2

a) Why syntax trees are important in translation

A syntax tree is a condensed, hierarchical representation of a program's structure that keeps only the operators, operands, and control constructs that carry meaning, unlike a parse tree, which records every grammar production used during parsing, including many nodes that add no semantic value. This compactness is important for translation for several reasons that map directly onto the student's problem.

- Because each internal node of the tree directly represents an operator, statement, or construct (such as a loop or an assignment), the code generator can walk the tree in postorder

and emit intermediate code for each node once its children have been processed — this is far simpler than trying to generate code straight from a deep parse tree, and it is the reason switching to syntax trees resolved the student's difficulty with nested loops and multi-operator expressions.

- Semantic analysis, including type checking, is performed by annotating the tree's nodes with attributes such as type and value while traversing it, so the same tree structure that supports code generation also supports the type checker the student added.
- Because the tree is smaller and free of redundant grammar detail, it is much easier to apply further processing such as constant folding or reordering to it than it would be to a full parse tree.
- The syntax tree captures the logical nesting of loops and expressions directly through parent-child relationships, so the translation scheme no longer has to fight the grammar's structure — this decoupling of parsing order from semantic structure is exactly what let the student move from a top-down, statement-by-statement scheme to a cleaner bottom-up evaluation over the tree.

b) How a type checker improves compiler reliability

A type checker examines every expression, assignment, and operator use and compares the type expressions of the operands against the language's type rules, using type-equivalence checking to decide whether two types are compatible. In the student's compiler this is exactly the missing piece: once variables of different types were used together, there was no component verifying whether the combination was legal.

- It catches type mismatches — such as assigning a floating-point value to an integer variable without conversion — at compile time, before the program is ever run, instead of letting the error surface as incorrect output or a crash at run time.
- It performs automatic type conversion for compatible pairs of types (for example, promoting an integer to a float in a mixed expression) while rejecting genuinely incompatible combinations, so the language stays flexible without becoming unsafe.
- It enforces consistent semantics across the whole program, so the same expression always behaves the same way regardless of where it appears.
- It gives precise, early error messages tied to the offending expression, which makes debugging far easier than diagnosing a wrong result after execution.

Taken together, these effects are what directly resolve the type-mismatch errors the student observed: a simple type checker with automatic conversions turns silent, inconsistent behaviour

into either a correctly converted value or a clear compile-time error, which is the core measure of a reliable compiler.

Question 3

a) The role of type systems in compiler design

A type system is the set of rules a language uses to assign a type to every expression and variable, and to decide which operations on those types are legal. In the robotics company's compiler, this role is central to solving the exact problem described: expressions mixing integers, booleans, and floating-point values were failing because no consistent rule existed for what combinations were allowed.

- It ensures semantic correctness by preventing operations that make no sense for the given types, such as combining a boolean sensor flag with a floating-point sensor reading without an explicit rule for how that should be interpreted.
- It provides type-equivalence checking — deciding, by name or by structure, whether two type expressions describe the same or compatible types — which lets the compiler tell legitimate mixed-type expressions apart from genuinely invalid ones.
- It supports automatic type conversion (coercion) for safe combinations, such as promoting an integer to a float, so the language remains convenient to use while unsafe combinations are still rejected.
- It enables errors to be detected at compile time rather than when a robot is executing a translated instruction in the field, which is critical where reliability and safety matter.
- It feeds directly into code generation, since the machine instruction chosen for an operation (for example, an integer add versus a floating-point add) depends on the type information the type system has already established.

Overall, the type system is what turns the robotics company's ad-hoc, error-prone handling of mixed data types into a systematic guarantee that only well-defined operations are translated into machine instructions.

b) How bottom-up evaluation assists syntax-directed translation

Bottom-up evaluation computes attributes in postorder, in step with how a bottom-up (LR/shift-reduce) parser recognises the input: by the time a production is reduced, the attribute values of every symbol on its right-hand side have already been computed, so the semantic action for that production can execute immediately. This is naturally an S-attributed style of evaluation, and it is why the robotics team's bottom-up approach worked well for generating syntax trees and intermediate representations directly during parsing.

- Each reduction step can build a syntax-tree node from the already-computed child nodes, so the tree is assembled incrementally as parsing proceeds, without a separate tree-construction pass.
- It requires only a single, efficient pass, since there is no need to build and separately traverse an attribute-dependency graph — the parser's own bottom-up order already respects the dependencies for synthesized attributes.
- It fits naturally with LR-family parsers (typically used by parser generators such as YACC), which the case implies the team is using for a domain-specific robotics language.
- It can compute the type of an expression as a synthesized attribute at the very same moment the corresponding tree node is created, so type equivalence and type conversion can be checked while the tree is being built — this is the mechanism that lets bottom-up evaluation directly resolve the incompatible-datatype problem described in the case.

Question 4

a) The significance of syntax-directed definitions in semantic analysis

A syntax-directed definition provides a formal, modular way of attaching semantic rules to a grammar so that meaning is computed as the syntax of the program is recognised. For the calculator-like language in this case, this significance shows up directly in the problem the team faced.

- It gives semantic analysis a systematic structure: every production carries the rules needed to compute the attributes of its symbols, so tasks such as building a syntax tree, checking types, or preparing for code generation are all expressed uniformly, attached to the grammar itself, rather than as separate ad-hoc code.
- It supports synthesized attributes for combining values bottom-up from operands (for example, computing the type or value of an expression from its sub-expressions) and inherited attributes for passing contextual information top-down (for example, an expected type or symbol-table entry), which together are exactly what is needed to process complex nested arithmetic correctly.
- It defines an attribute-dependency graph that fixes a valid evaluation order, so that every attribute a rule needs is guaranteed to already be available — this is what was missing when the compiler failed to generate correct syntax trees for nested arithmetic operations, since without an explicit dependency order, evaluation could be attempted before the required values existed.

- It produces a fully annotated syntax tree that carries all semantic information forward, so later phases such as type checking and type-equivalence checking can operate directly on the tree the SDD produced.

b) How type equivalence helps prevent semantic errors

Type equivalence is the rule the compiler uses to decide whether two type expressions represent the same, or a compatible, type — either by name equivalence (the declared type names must match) or by structural equivalence (the underlying structure of the types must match, regardless of name).

- It lets the compiler detect, precisely, when two type expressions being compared or assigned are not compatible — which is exactly the error the calculator-language team was seeing when variables with incompatible type expressions were compared and assigned incorrectly.
- It allows the compiler to flag an illegal operation with a specific, meaningful error at the point it occurs, instead of allowing an inconsistent value to be produced and propagated silently through later expressions.
- It works together with implicit type conversion: when two types are not identical but are convertible under the language's rules (for example, integer to floating-point), the compiler inserts the conversion automatically; when they are not convertible at all, an error is reported instead.
- By applying this check consistently at every comparison and assignment, type equivalence keeps the semantics of nested arithmetic expressions predictable and correct, which is the reliability improvement the team was aiming for.

Question 5

a) Comparing top-down and bottom-up translation methods

Top-down translation evaluates semantic rules while the parser expands the grammar from the start symbol downward, typically using recursive-descent or LL parsing. It naturally supports inherited attributes, since context (such as an expected type, or a variable's scope) can simply be passed down as a parameter of the recursive call that processes each sub-expression. This worked for the startup's simple statements, but as the case describes, it struggled once statements became deeply nested, because the number of inherited values that had to be threaded through the recursive calls grew with the nesting depth, and left-recursive grammar rules — common for repeated statements and expressions — cannot be handled directly by top-down parsing.

Bottom-up translation evaluates semantic rules as the parser reduces the input upward toward the start symbol, typically using LR/shift-reduce parsing. It naturally supports synthesized attributes,

since by the time a production is reduced, the values of its right-hand-side symbols are already available, so attribute values and syntax-tree nodes can be combined immediately. It also handles left-recursive grammars comfortably, which is important for lists of statements and for expression grammars.

Basis	Top-Down Translation	Bottom-Up Translation
Parsing style	Recursive-descent / LL parsing, expands the start symbol downward	Shift-reduce / LR parsing, reduces input upward to the start symbol
Attribute support	Naturally supports inherited attributes (L-attributed definitions)	Naturally supports synthesized attributes (S-attributed definitions)
Handling of nested statements	Becomes difficult as attribute dependencies grow with nesting depth, as seen in this case	Handles nested statements more efficiently since values are combined as soon as a production is reduced
Left recursion	Cannot handle left-recursive grammars directly	Handles left-recursive grammars naturally
Syntax tree construction	Tree built top-down while descending the grammar	Tree built bottom-up as sub-trees are combined during reduction
Efficiency for this case	Less efficient once expressions/statements are deeply nested	More efficient; adopted by the team to resolve attribute-dependency difficulties

For the startup's scripting language, adopting bottom-up evaluation together with syntax-tree construction is what resolved the difficulty, because it removed the need to carry a growing set of inherited values through deeply nested recursive calls, and instead let each nested statement's translation be completed and combined into its parent as soon as it was reduced.

b) Why type checking and type conversion are essential in modern compilers

Modern languages such as the startup's scripting language routinely mix several data types — in this case arrays, integers, and floating-point variables — within the same statements, so the compiler needs a systematic way to keep those combinations correct.

- Type checking verifies that every operation and assignment respects the language's type rules, catching illegal combinations — such as treating an array as if it were a scalar, or mixing incompatible numeric types without a defined rule — at compile time rather than allowing them to cause incorrect results or failures once the program runs.
- Type conversion (coercion) allows compatible types to be combined safely — for example promoting an integer to a floating-point value in a mixed arithmetic expression — so that the language remains convenient and expressive without weakening safety.
- Together, they prevent semantic ambiguity between variables of different types, so the same expression cannot silently mean two different things depending on how its operands happen to be typed.

- They make the intermediate code produced by the compiler more portable and predictable, since the conversions applied are explicit and consistent rather than left to the behaviour of a particular target platform.
- Overall, they give the programmer and the compiler user confidence that a program which compiles successfully will behave consistently — which is precisely the reliability the startup's type checker, verifying type expressions and performing safe conversions, was designed to deliver.